2024 17th International Congress on Image and Signal Processing, BioMedical Engineering and Informatics (CISP-BMEI) | 979-8-3315-0739-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/CISP-BMEI64163.2024.10906132

Priority-Aware Task Offloading and UAV Trajectory Optimization for Aerial Access Network-Assisted Mobile Edge Computing

Yuqiang Zhou [1*] , Haifeng Sun [1†] .

[1] School of Computer Science and Technology, Southwest University of Science and Technology
zhouyuqiang@mails.swust.edu.cn, dr_hfsun@163.com

***Abstract*—With the surge in computational data, Mobile Edge Computing (MEC) is set to become a crucial technology for reducing communication latency and congestion. However, the widespread adoption of MEC faces several challenges. Aerial Access Networks (AANs), comprising hierarchical High Altitude Platforms (HAPs) and low-altitude Unmanned Aerial Vehicles (UAVs), offer a groundbreaking framework for MEC task offloading, particularly enhancing the service experience of Internet of Things (IoT) devices in disaster zones, battlefield healthcare, or remote areas. In this paper, we propose an MEC task offloading framework supported by AANs to serve IoT devices distributed on the ground. We define system gain based on the energy consumption and latency of tasks with varying priorities. Our objective is to maximize workload fairness among UAVs and overall system gain by jointly optimizing UAVs' flight trajectories, IoT devices' computational task offloading decisions, and service fairness. We propose a multi-agent proximal policy optimization (MAPPO)-based algorithm to solve this joint optimization problem. Experimental results validate the effectiveness of the proposed approach, and numerical analysis evaluates system performance.**

***Index Terms*—Mobile Edge Computing, task offloading, Aerial Access Networks, joint optimization, MAPPO**

I. Introduction

With the rapid development of mobile internet services and the continuous advancements in Internet of Medical Things (IoMT) technology, there has been a significant increase in the communication bandwidth requirements for healthcare systems. To meet the Quality of Service (QoS) requirements for medical Internet of Things(IoT) devices, such as wearables and remote patient monitoring systems, a network with low latency, high robustness, and reliability is essential.

Mobile Edge Computing (MEC) has been proposed as a solution to enhance resource-limited medical IoT devices. Through MEC, compute-intensive tasks, such as medical image processing and large-scale bioinformatics analyses, can be offloaded to nearby MEC servers, significantly improving computational speed and reducing the energy consumption of medical devices. However, in critical environments such as disaster response or battlefield healthcare, deploying fixed wireless Access Points with MEC services faces challenges due to the difficulty and cost of setup, along with the inability to dynamically replace faulty equipment. Thus, improving the robustness and flexible deployment capabilities of MEC networks is essential for the development of reliable and efficient healthcare systems.

Aerial Access Networks (AANs) are emerging as effective solutions for Mobile Edge Computing (MEC) by utilizing both High Altitude Platforms (HAPs) and low-altitude Unmanned Aerial Vehicles (UAVs) in a complementary manner [1], [2]. HAPs, positioned at altitudes of around 20 kilometers and covering diameters of 60 to 80 kilometers, serve as stable airborne base stations, equipped with powerful payloads like MEC servers and communication equipment. With their extended endurance of several months, HAPs provide broad coverage and auxiliary computational offloading services, addressing the coverage limitations of UAVs [3], [4]. UAVs, on the other hand, act as flying base stations, equipped with MEC servers, communication systems, and storage units, dynamically adjusting their positions to meet offloading demands from ground IoT devices. Leveraging hover stability and line-of-sight transmission, UAVs ensure high-speed, low-latency wireless access and MEC services through air-to-ground (A2G) links, even in areas lacking traditional network infrastructure [5], [6]. Together, the hierarchical structure of HAPs and UAVs forms a robust and flexible MEC infrastructure, supporting ground IoT devices with efficient computational offloading and network access capabilities.

Compared with traditional MEC frameworks that rely on cellular access points (APs) or UAVs, the MEC frameworks supported by AANs offer advantages in terms of flexibility, coverage areas, and QoS. Since the offloaded tasks are processed by different UAVs in AANs, the dynamic, decentralized, and multi-agent nature of AAN-assisted MEC frameworks align well with the capabilities of Multi-agent Markov Decision Process (MAMDP), making it a suitable approach for modeling and optimizing decision-making processes [7].

Extensive research has been dedicated to tackling the joint optimization MAMDP problem in MEC offloading, leveraging deep reinforcement learning (DRL) to facilitate intelligent decision-making in high-dimensional continuous action spaces. Multi-agent DRL methods, such as the Multi-agent Twin Delayed Deep Deterministic Policy Gradient (MATD3) [8] and the Multi-agent Deep Deterministic Policy Gradient

* Contribute equally to this work, † Corresponding author. Haifeng Sun: dr_hfsun@163.com.

979-8-3315-0739-8/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:37:36 UTC from IEEE Xplore. Restrictions apply.

(MADDPG) [9], have been developed. MADDPG incorporates deep neural networks, allowing it to manage high-dimensional, non-linear state, and action spaces effectively. Today, MADDPG is extensively utilized to tackle collaborative and cooperative challenges in MEC task offloading, offering a flexible and efficient framework for optimizing multi-agent problems.

Inspired by aforementioned discussions, we explore a hierarchical AAN framework consisting of a high altitude HAP and multiple low-altitude UAVs, aiming to provide optimized offloading service for ground IoT devices. Our objective is to maximize workload fairness for UAVs and overall system gain, by jointly optimizing UAVs' flight trajectories, together with IoT devices' computational task offloading decision and service fairness. In order to solve the non-convex optimization problem, we first transform it into MAMDP. Subsequently, We propose a MAPPO-based algorithm by implementing the independent agent on each UAV and the centralized training critic network on the HAP.

Subsequent parts of this manuscript are structured thusly: Section II elaborates on the proposed framework and delineates the corresponding problem statement. A comprehensive exposition of our methodological approach is presented in Section III. Section IV furnishes numerical results along with a performance assessment. The paper culminates with concluding remarks in Section V.

II. System Model And Problem Formulation

A. System Description

The mobile edge computing system, designed in a hierarchical AAN framework as shown in Fig. 1, consists of N ground IoT devices within a square area with the edge length of l^{max}. The IoT device is represented as $n \in \mathcal{N} = \{1, 2, \ldots, N\}$. Above this area, a set of UAVs is flying at the height of H, with each one is denoted as $m \in \mathcal{M} = \{1, 2, \ldots, M\}$ [8]. Additionally, within this area, a fixed-position HAP denoted as K, is hovering at an altitude of H_K ($H_K >> H$).

For each time slot t in the discrete time system denoted by $t \in \mathcal{T} = \{1, 2, \ldots, T\}$ with the time length τ, each resource constrained IoT device will generate a computation-intensive task, which can only be executed locally, or fully offloaded to either a UAV or the HAP for computing. Let $\mathcal{M}' = \{0, 1, 2, \ldots, M+1\}$ be a collection of possible locations where tasks can be executed. Thus, the offloading decision variable for IoT device n at the t-th time slot is represented by the following equation

$$\alpha_{n,m,t} = \{0, 1\},\ m \in \mathcal{M}'. \tag{1}$$

For the task generated on the IoT device n, if it is optimized to be processed locally, then we get $\alpha_{n,0,t} = 1$, and $\alpha_{n,m,t} = 0$ with $m \neq 0$. If the task is offloaded to the HAP, we get $\alpha_{n,M+1,t} = 1$, thus $\alpha_{n,m,t} = 0$ with other values of m. While the task is offloaded to a UAV m, we get $\alpha_{n,m,t} = 1$, and $\alpha_{n,m,t} = 0$ with other values of m. Consequently, in the t-th time slot, the decision variable associated with IoT device n can be expressed in the following

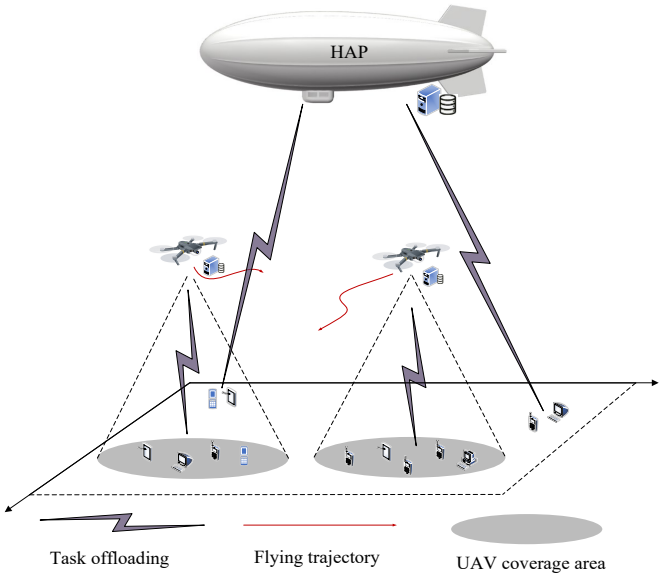


Fig. 1. System Framework

$$\sum_{m \in \mathcal{M}'} \alpha_{n,m,t} = 1. \tag{2}$$

In each time slot, each IoT device has a computation-intensive task $L_{n,t}$, which is defined as

$$L_{n,t} = (C_{n,t}, D_{n,t},\ delay_{n,t}), \tag{3}$$

where $D_{n,t}$ represents the task data size, $C_{n,t}$ denotes the number of CPU cycles, $delay_{n,t}$ indicates the maximum tolerable delay for the task.

B. UAV Trajectory Control Model

At each time slot t, we assume that the HAP's location remains fixed at $q_{K,t} = [X_{K,t}, Y_{K,t}, H_K]$. Meanwhile, for each UAV, its flight position is determined by the flight angle $\theta_{m,t} \in [0, 2\pi)$ and the distance $d_{m,t} \in [0, d_{\max}]$. The initial coordinates of each UAV are set to $q_{m,0} = [X_{m,0}, Y_{m,0}, H]$. As a result, the UAV's coordinates at time slot t can be expressed as $q_{m,t} = [X_{m,t}, Y_{m,t}, H]$, where $X_{m,t} = X_{m,0} + \sum_{t'=1}^{t} d_{m,t'} \cos(\theta_{m,t'}), 0 \leq X_{m,t} \leq l^{max}$, and $Y_{m,t} = Y_{m,0} + \sum_{t'=1}^{t} d_{m,t'} \sin(\theta_{m,t'}), 0 \leq Y_{m,t} \leq l^{max}$ [8].

To avoid collisions during flights, UAVs should maintain a minimum distance of R_{min}. Therefore, we have

$$\|q_{m,t} - q_{m',t}\| \geq R_{min}, \forall m, m' \in \mathcal{M}, m \neq m'. \tag{4}$$

Assuming the coordinates of the n-th IoT device are given by $q_{n,t} = [X_{n,t}, Y_{n,t}, 0]$, the distance between the IoT device n and the UAV m at time slot t is defined as

$$R_{m,n,t} = \|q_{m,t} - q_{n,t}\|, \forall m \in \mathcal{M}. \tag{5}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:37:36 UTC from IEEE Xplore. Restrictions apply.

Simultaneously, the distance between the IoT device n and the HAP is defined as

$$R_{K,n,t} = \|q_{K,t} - q_{n,t}\|. \tag{6}$$

We suppose the service area of the HAP can cover all the IoT devices in the proposed scenario due to its higher altitude and stronger communication capability, but due to the limited service range of UAVs, the IoT devices only located within the service area covered by the UAV can get its offloading service. Following the approach in literature [10], we introduce ψ as the minimum elevation angle of the UAV from an IoT device. Then, we have

$$\alpha_{n,m,t} R_{m,n,t} \cos(\psi) \leq R_{max}, \forall m \in \mathcal{M}, \tag{7}$$

where $\psi = \frac{\pi}{4}$ and R_{max} represents the maximum radius of horizontal ground coverage for the UAV.

C. Communication Model

It is reasonable to assume that there are line-of-sight communication links between UAVs and IoT devices, as well as between the HAP and IoT devices [11]. Based on this, the channel power gains can be calculated according to the distance between UAVs and IoT devices using a freespace path-loss model [12], [13]. As a result, the channel power gain between UAV m and IoT device n during time slot t follows this model

$$g_{m,n,t} = \ g_0 R_{m,n,t}^{-2} = \frac{g_0}{\| \ q_{m,t} - q_{n,t} \ \|^2}, \forall m \in \mathcal{M}, \tag{8}$$

where g_0 denotes the channel power gain at a reference distance of 1 meter.

To prevent transmission interference when multiple IoT devices offload tasks to the same UAV, we assume that ground IoT devices use orthogonal frequency division multiple access (OFDMA) to communicate with their respective UAVs [14], [15]. The data rate for the communication channel between IoT device n and UAV m is computed as follows

$$r_{m,n,t} = B_n \log_2 \left(1 + \frac{P_n g_{m,n,t}}{\sigma_0^2}\right), \forall m \in \mathcal{M}, \tag{9}$$

where B_n represents the communication bandwidth between the IoT device and the UAV, P_n indicates the transmission power of the IoT device in the uplink, and σ_0^2 represents the noise power.

Similarly to the IoT devices and UAVs channel, the channel power gain between IoT device n and the HAP at time slot t is computed as follows

$$g_{K,n,t} = \ g_0 R_{K,n,t}^{-2} = \frac{g_0}{\| \ q_{K,t} - q_{n,t} \ \|^2}. \tag{10}$$

Then, the wireless transmission rate from the IoT device n to the HAP is denoted as

$$r_{K,n,t} = B_n \log_2 \left(1 + \frac{P_n g_{K,n,t}}{\sigma_0^2}\right). \tag{11}$$

D. Task Processing Model

Each IoT device has the option to process the task $L_{n,t}$ locally, offload it to a UAV, or remotely offload it to the HAP. The detailed information regarding task processing in different locations is listed below.

1) Local Computation: Assuming that the IoT device performs the computation locally, the computation delay required to complete task $L_{n,t}$ is as follows

$$T_{n,0,t} = \frac{C_{n,t} D_{n,t}}{f_n}, \tag{12}$$

where f_n denotes the clock frequency at which IoT device n operates. The energy required to execute this task is

$$E_{n,0,t} = k_n (f_n)^{v_n} T_{n,0,t}, \tag{13}$$

where k_n is the effective switched capacitor, $v_n > 1$.

2) Offloading to UAV: If the computation-intensive task $L_{n,t}$ is offloaded to be computed on UAV m, $\forall m \in \mathcal{M}$, from the device n at time slot t, he computation delay required to complete task $L_{n,t}$ is as follows

$$T_{n,m,t} = \frac{D_{n,t}}{r_{m,n,t}} + \ \frac{C_{n,t} D_{n,t}}{f_m}, \tag{14}$$

where f_m refers to the clock frequency of the UAV. Since the output data size is minimal, the task retrieval time is disregarded.

The corresponding offloading energy consumption for IoT device n is

$$E_{n,m,t} = P_n \frac{D_{n,t}}{r_{m,n,t}}. \tag{15}$$

3) Offloading to HAP: If the computation-intensive task $L_{n,t}$ is offloaded to be processed on the HAP at time slot t, the required processing delay includes time for both data offloading and computation on the HAP as

$$T_{n,M+1,t} = \frac{D_{n,t}}{r_{K,n,t}} + \ \frac{C_{n,t} D_{n,t}}{f_K}. \tag{16}$$

Similarly, the corresponding offloading energy consumption required by IoT device n is

$$E_{n,M+1,t} = P_n \frac{D_{n,t}}{r_{K,n,t}}. \tag{17}$$

So, the total processing delay and energy consumed by IoT device n to finish a task are respectively derived as

$$T_{n,m,t}^{sum} = \sum_{m \in \mathcal{M}'} \alpha_{n,m,t} \ T_{n,m,t}, \tag{18}$$

and

$$E_{n,m,t}^{sum} = \sum_{m \in \mathcal{M}'} \alpha_{n,m,t} \ E_{n,m,t}. \tag{19}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:37:36 UTC from IEEE Xplore. Restrictions apply.

E. Task Priority Model

In real-world communication and computation environments, tasks may differ in terms of their priorities [16]. Let $Pr_{n,t}$ represent the priority of a task for IoT device n, where $Pr_{n,t} = 1$ indicates a high-priority task and $Pr_{n,t} = 0$ denotes a low-priority task. There are also medium-priority tasks, which are assigned values in the range of $Pr_{n,t} \in [0, 1]$. To avoid the issue of low-priority tasks being starved of computational resources, we use a utility function that takes into account priority, task completion time, and allowable delay thresholds, rather than relying on preemptive scheduling approaches. Using this approach, the utility function for high-priority tasks is formulated

$$U^H_{n,m,t} = \begin{cases} log_2\left(1 + delay_{n,t} - T^{sum}_{n,m,t}\right), & T^{sum}_{n,m,t} \le delay_{n,t} \\ -P^H, & T^{sum}_{n,m,t} > delay_{n,t} \end{cases} \tag{20}$$

where constant P^H represents a negative penalty for failing to complete high-priority tasks within the set delay thresholds.

For tasks assigned a low priority, the requirements regarding completion time are fairly flexible. If these tasks are not completed within the specified delay limit, their utility gradually decreases at an exponential rate, even though they are still considered. However, when low-priority tasks are finished before the designated deadline, they are rewarded with a fixed value. Based on these principles, we develop the utility function for such tasks

$$U^L_{n,m,t} = \begin{cases} P^L, & T^{sum}_{n,m,t} \le delay_{n,t} \\ P^L e^{-\rho\left(T^{sum}_{n,m,t} - delay_{n,t}\right)}, & T^{sum}_{n,m,t} > delay_{n,t} \end{cases} \tag{21}$$

where constant P^L is a fixed positive reward for completing low-priority tasks on time, with $\rho > 0$ as the exponential decay factor.

In order to measure the utility of mid-priority task processing [17], a linear interpolation approach is applied to the IoT device n, this is

$$U_{n,m,t} = Pr_{n,t} U^H_{n,m,t} + (1 - Pr_{n,t})\, U^L_{n,m,t} \tag{22}$$

F. Workload Fairness Model

To address the issue of uneven workload distribution during UAV service, where certain UAVs handle significantly more IoT devices than others, we propose a solution. Some UAVs are burdened with serving a majority of IoT devices, leading to a considerably heavier workload compared to those UAVs that cater to fewer devices. To mitigate this imbalance, we introduced $\hat{F}_{m,t} = \sum_{t'=1}^{t} \sum_{n=1}^{N} \alpha_{n,m,t'}$ to represent the number of IoT devices served by UAV m from time t' to t. Consequently, the workload fairness index f^U_t, which quantifies the degree of workload fairness across UAVs, can be formulated as

$$f^U_t = \frac{\left(\sum_{m=1}^{M} \hat{F}_{m,t}\right)^2}{M \sum_{m=1}^{M} \left(\hat{F}_{m,}\right)^2}. \tag{23}$$

If all UAVs experience a uniform IoT workload from the beginning up to time slot t, the value of f^U_t will asymptotically approach 1.

G. Problem Formulation

Following this, we describe the system gain utility function, which is defined as follows

$$\boldsymbol{H}_t = f^U_t \sum_{m=1}^{M} \sum_{n=1}^{N} (\omega_1 U_{n,m,t} - \omega_2 E^{sum}_{n,m,t}) \tag{24}$$

where ω_1 and ω_2 are weight parameters.

The joint optimization problem involving flight trajectory planning and task offloading decisions for IoT devices is formulated as follows

$$\max_{\alpha_{n,m,t}, \theta_{m,t}, d_{m,t}} \sum_{t \in \mathcal{T}} \boldsymbol{H}_t \tag{25a}$$

$$s.t. (1), (2), (4), (7), \tag{25b}$$

$$0 \le \theta_{m,t} < 2\pi, \forall m \in \mathcal{M}, t \in \mathcal{T}, \tag{25c}$$

$$0 \le d_{m,t} \le d_{max}, \forall m \in \mathcal{M}, t \in \mathcal{T}, \tag{25d}$$

The optimization problem's combination of discrete and continuous variables makes it difficult to solve using conventional mathematical optimization techniques, given its NP-hard complexity. To tackle this issue, we introduce a MAPPO-based approach, a distributed reinforcement learning algorithm designed specifically for this challenge.

III. Approach Design

To achieve the optimal solution, decisions involving $\alpha_{n,m,t}$, $d_{m,t}$, and $\theta_{m,t}$ are made by decoupling UAV trajectory planning and IoT task offloading. In each time slot, IoT devices compare potential offloading locations and select the most suitable one, which affects the reward in the central critic network and guides UAV actions.

A. Offloading Decision

To prevent certain IoT devices from being continuously served while others remain unattended for extended periods, the provision of services to IoT devices should maintain fairness in service delivery [18]. We define $\hat{F}_{n,t} = \{0, 1\}$ to indicate whether the task generated by IoT device n during time slot t is served. If the task is served, its value is 1, and if not, it is 0. The service fairness constraint f^I_t for all IoT devices at time slot t is defined as

$$\begin{aligned} f^I_t &= \mu_n - \varphi \cdot \sigma_n \\ &= \frac{\sum_{n=1}^{N} \sum_{t'=1}^{t} \hat{F}_{n,t'}}{N} \\ &\quad - \varphi \sqrt{\frac{\sum_{n=1}^{N} \left(\sum_{t'=1}^{t} \hat{F}_{n,t'} - \frac{\sum_{n=1}^{N} \left(\sum_{t'=1}^{t} \hat{F}_{n,t'} \right)}{N} \right)^2}{N-1}}, \end{aligned} \tag{26}$$

where μ_n and σ_n denote the mean and standard deviation of the service times for all IoT devices, respectively. We introduce a service fairness factor φ, $0 \le \varphi \le \frac{\mu_n}{\sigma_n}$, to control how

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:37:36 UTC from IEEE Xplore. Restrictions apply.

aggressive the service fairness approach is. As φ nears $\frac{\mu_n}{\sigma_n}$, the service fairness constraint f_t^I approaches zero, indicating that no IoT devices are unfairly served.

Additionally, we define $b_{n,t} = \{0, 1\}$ to indicate whether IoT device n is unfairly served at time t. If its total service count, $\sum_{t'=1}^{t} \hat{F}_{n,t'}$, falls below f_t^I, the device is considered unfairly served (i.e., $b_{n,t} = 1$); otherwise, $b_{n,t} = 0$. Any IoT device identified as being unfairly served has the option to offload its task $L_{n,t}$ to the HAP for computational assistance.

We design a low-complexity method to optimize device offloading decision. Define $\mathcal{M}^*$ as the set of potential locations in $\mathcal{M}'$ that do not include the HAP. Consequently, for each IoT device n at time slot t, the offloading decision is determined using the following expression

$$\alpha_{n,m,t} = \begin{cases} 1 \ and \ m = \underset{m \in \mathcal{M}^*}{\arg\max} U_{n,m,t}, & \text{if } b_{n,t} = 0, \\ 1 \ and \ m = \underset{m \in \mathcal{M}'}{\arg\max} U_{n,m,t}, & \text{if } b_{n,t} = 1, \\ 0, \ otherwise. \end{cases} \quad (27)$$

Specifically, when $b_{n,t}$ is 0, it indicates that the IoT device n is fairly served, and can only choose the most suitable place for execution from the set $\mathcal{M}^*$. On the other hand, $b_{n,t}$ is 1 signifying an unfairly served device of n, the device can choose the most suitable location for execution from the set $\mathcal{M}'$.

B. UAV Trajectory

Considering that UAV movements can impact the system's environmental state, the overall system cost is influenced by both the environment's current condition and the combined actions of all UAVs. Additionally, past system states and previous actions together result in the environment transitioning into new random states [19]. In this scenario, the task offloading optimization problem can be modeled as a MAMDP $\left\langle \mathcal{M}, \mathcal{S}, \mathcal{A}, \mathcal{P}, \{\mathcal{R}_m\}_{m \in \mathcal{M}} \right\rangle$, where $\mathcal{M}$ denotes the set of agents, $\mathcal{S} = \{s_t, t \in \mathcal{T}\}$ represents the state set of all agents, and $\mathcal{A} = \{a_t, t \in \mathcal{T}\}$ defines the action space. Here, $\mathcal{P}$ stands for the state transition probability, and $\mathcal{R}_m = \{r_{m,t}, t \in \mathcal{T}\}$ is the reward function for the agents. Each state s_t is composed of private observations $s_{m,t}$ and additional information available to each agent. A dedicated agent controls each UAV, and during every time slot, each UAV selects an action $a_{m,t}$ based on its private observation $s_{m,t}$. Afterward, each UAV receives a corresponding reward $r_{m,t}$. The collective actions of all UAVs cause a global state transition from the current state s_t to the subsequent state s_{t+1}, governed by the transition probability $\mathcal{P}$.

Thus, we define the observations, actions, and reward functions of each intelligent agent in time slot t as follows:

1) Observation $o_{m,t}$: The observation at time encompasses various crucial components describing the state relate to UAVs, IoT devices and the HAP, which is defined as $o_{m,t} = \{L_{n,t}, q_{n,t}, q_{m,t}, R_{m,n,t}, f_t^U\}$
2) Action $a_{m,t}$: We define the action as $a_{m,t} = \{\theta_{m,t}, d_{m,t}\}$, representing the UAV's flight direction, flight distance in the *t*-th time slot.
3) Reward $r_{m,t}$: According to the joint optimization problem,the reward function $r_{m,t}$ for UAV m is defined as the system gain (25a).

$$r_{m,t} = \begin{cases} \boldsymbol{H}_t + \eta_1, & \text{if SC}, \\ \boldsymbol{H}_t - \eta_2 - \eta_3, & \text{otherwise}. \end{cases} \quad (28)$$

where SC indicates that the constraints are satisfied, η_1 is an extra bonus; η_2 and η_3 represent penalties associated with boundary constraints and collision constraints, respectively. If a UAV fails to meet the boundary constraints, it incurs a penalty (η_2). Furthermore, if any UAV collision occurs, both UAVs involved are penalized (η_3).

Moreover, since MAPPO directs each agent to select actions that benefit the entire system, rather than only itself, all UAVs share a common system reward r_t. In a multi-agent learning environment, the state transition of the system is determined by the combined actions of all agents. Thus, the overall environment state is represented as $s_t = \{o_{m,t}, \forall m \in \mathcal{M}\}$, and the collective system action is defined as $a_t = \{a_{m,t}, \forall m \in \mathcal{M}\}$. Within a cooperative multi-agent reinforcement learning framework, the maximum expected discounted reward is attained through the total sum of rewards from all UAVs, which is expressed as

$$r_t = \sum_{m=1}^{M} r_{m,t}. \quad (29)$$

C. Multi-Agent DRL Algorithm

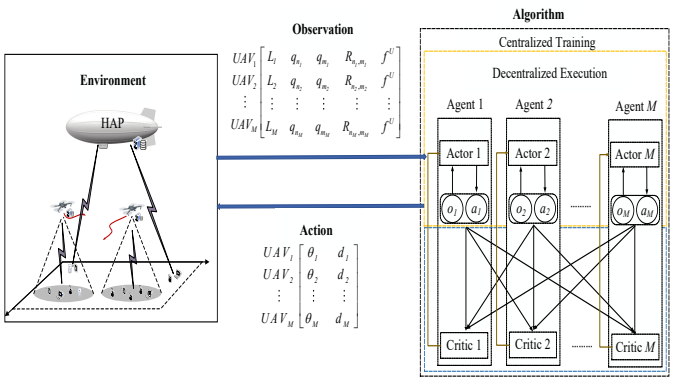


Fig. 2. The framework of MAPPO.

To solve the aforementioned MAMDP, given the high-dimensional continuous action space in the task offloading optimization problem, a MAPPO-based algorithm is proposed, as shown in Fig. 2 [2]. The MAPPO framework consists of M agents interacting with the environment, where each UAV operates as an agent. Each agent in MAPPO trains two separate neural networks: one is the actor network, and the other is the value function network (referred to as the critic). Each

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:37:36 UTC from IEEE Xplore. Restrictions apply.

agent functions in two distinct phases: centralized training and decentralized execution. During the offline training phase, exploration is employed to discover the optimal policy. In the execution phase, only forward propagation is needed, with no requirement for random exploration. To illustrate this process, we will use an agent as an example to explain how centralized training is conducted in the MAPPO model and how it operates during decentralized execution.

During the initial stage of centralized learning, a coordinating entity known as the critic functions to determine the unified action-value function. This function, denoted as $Q_m(s, a_1, a_2, ..., a_m|\phi)$, incorporates the comprehensive state information, encompassing each agent's behaviors and perceptions. This centralized Q function evaluates the actions of the actors from a holistic perspective, directing them toward selecting more optimal actions [2]. The critic network then refines its parameters θ^c by minimizing the loss function

$$\text{loss}(\theta^c) = E\left[Q_m(s, a_1, a_2, ..., a_m|\phi)) - y^{MA}\right]^2 \quad (30)$$

where

$$y^{MA} = r_t + \gamma Q'_m(s'_t, a'_1, a'_2, \ldots, a'_m|\phi')$$

in which $s = (s_1, s_2, ..., s_m)$ represents the state vector, ϕ is the parameter used to calculate the Q-value for the current critic network, and θ^c is the set of parameters updated during training. s' denotes the updated states for the target network, while ϕ' represents the revised parameters of the evaluation network. Simultaneously, the actor network updates its parameters θ and chooses actions based on the centralized Q function from the critic network and its own observations. Specifically, the actor network adjusts θ in the direction indicated by $\nabla_\theta J(\theta)$, as demonstrated in equation (31). Incorporating a global state transforms the partially observable MDP into a fully observable MDP. This transformation simplifies and accelerates value learning by leveraging a centralized value function [20]

$$\nabla_\theta J(\theta) \approx E[\nabla_a Q(s, a_1, a_2, \ldots, a_m|\phi)\nabla_\theta \pi(s|\theta)]. \quad (31)$$

In the phase of decentralized execution, the critic network remains inactive, while the pre-trained actor network operates autonomously. Decisions made by the decentralized agents rely exclusively on the UAVs' local observations [21]. This phase is characterized by a straightforward forward propagation mechanism, which substantially reduces both time and computational resources compared to the training phase. With fine-tuned parameters, each agent can achieve a nearly optimal action globally without requiring information from other agents. Algorithm 1 presents the pseudocode for the method proposed.

Algorithm 1 MAPPO-Based Optimization Approach for Integrating Workload Fairness and System Gain.

Initialize policy network π_θ and value network V_ϕ, let $\pi_{\theta'} = \pi_\theta$
Initialize the experience replay buffer D;
for Episode = 1, 2, ..., L **do**
  s_1 = initialize observation
  for t = 1 to T **do**
    for UAV $m \in \mathcal{M}$ **do**
      Obtain action $a_{m,t}$
      Execute action $a_{m,t}$. If a UAV flies out of the target area or collides with another UAV, the UAV will remain in its current position.
    end for
    Calculate f_t^I
    for IoT devices $n \in \mathcal{N}$ **do**
      Obtain the offloading decision $\alpha_{n,m,t}$ according to Eq. (27)
    end for
    for UAV $m \in \mathcal{M}$ **do**
      Obtain $r_{m,t}$ according to Eq. (28)
      Obtain $s_{m,t+1}$
    end for
    Get the next state s_{t+1}
  end for
  Compute $\{Q^m(s_t, a_t)\}_{t=1}^T$ according to Eq. (25)
  Compute $\{A^m(s_t^m, a_t)\}_{t=1}^T$ according to Eq. (29)
  Store transition $\{o_{m,t}, a_{m,t}, Q^m(s_t, a_t), A^m(s_t^m, a_t)\}$ into D
  for k = 1, 2, ..., K **do**
    Shuffle the data's order and renumber
    for j = 1, 2, ..., $T/B - 1$ **do**
      Select B group of data $\{o_{m,t}, a_{m,t}, Q^m(s_t, a_t), A^m(s_t^m, a_t)\}_{i=1+Bj}^{B(j+1)}$
      for UAV $m \in \mathcal{M}$ **do**
        compute gradient according to Eq. (30) and Eq. (31)
        Apply gradient ascent on θ^m using by Adam
        Apply gradient ascent on ϕ^m using by Adam
      end for
    end for
  end for
  Update the network parameters for all UAV.
  Empty D
end for

IV. Performance Evaluation

In this section, we first outline the simulation settings. Then, we present the convergence performance of the algorithm and evaluate the efficacy of our approach.

A. Simulation Settings

This section demonstrates the effectiveness of the proposed MAPPO method for addressing the joint optimization problem, as evidenced by simulation results. We conducted 16,000 episodes, each consisting of 24 steps. The simulation system is built with Python 3.9, utilizing the PyTorch machine learning framework. Both the actor and critic networks are designed with three fully connected hidden layers, having neuron configurations of [400, 256, 128]. Both networks are trained using a learning rate of 1×10^{-5}.

The target area is defined as a square region of 1 km × 1 km, with 45 IoT devices within the area. We use 1 HAP and 2 UAVs to process tasks generated by IoT devices in the target area. The position of the HAP is fixed at [500m, 500m, 20km],

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:37:36 UTC from IEEE Xplore. Restrictions apply.

while the UAVs are initially positioned at [100m, 100m, 200m] and [100m, 900m, 200m], respectively. Furthermore, each device produces one task per time slot, with a time duration of $\tau = 1s$. Additional key simulation parameters are outlined in Table I.

TABLE I
SIMULATION PARAMETERS

Parameter	Value	Parameter	Value
N	$\{20, 25, ..., 45\}$	B_n	10 MHz
l_{max}	1000 m	f_n	0.5GHz
R_{max}	200 m	f_m	1.2GHz
$C_{n,t}$	[1000,2000] cycles/bit	f_K	5GHz
$D_{n,t}$	[10,15] Kb	σ_0^2	10^{-12} Watt
P_n	0.1 Watt	R_{min}	20 m
g_0	1.42×10^{-4}	γ	0.95

To evaluate the effectiveness of the MAPPO-Based for UAV path optimization, we compared it with two other methods using the same IoT task offloading strategy outlined in Eq. (27).

1) MADDPG : This widely adopted and dependable approach for multi-agent reinforcement learning incorporates paired decision-making and evaluation components. The decision-generating network produces action estimates, introduces exploratory perturbations, while its counterpart assesses the value associated with chosen actions.

2) MATD3 : It modifies TD3 for multi-agent contexts by incorporating centralized training while maintaining decentralized execution. Each agent's value function considers all agents' observations and actions, while its policy relies only on its own observations. MATD3 improves on MADDPG by using twin Q networks, and selecting the minimum value to reduce overestimation bias.

B. Simulation Results Analysis

First, we show the training curves of the MAPPO, MATD3, and MADDPG algorithms in a scenario involving two UAVs in Fig. 3. As shown in Fig. 3, it is clear that the initial cumulative reward stays below 0 and gradually increases with the number of training sessions. After about 6000 training sets, the curve stabilizes, indicating convergence of the training process. The results show that our employ MAPPO outperforms MATD3 and MADDPG.

Fig. 4 shows a heat map displaying the varying service times for each IoT device (with different color shadings in the color bar representing different time periods) and the UAV trajectories generated based on MAPPO. The central cross in the figure marks the position of the HAP. Analysis indicates that the UAVs effectively collaborate to cover the target areas as proposed in the scheme. Additionally, we compared the impact of different φ values on the service count of IoT devices outside the UAV service range. When $\varphi = 5$, the service count for IoT devices is significantly lower than when $\varphi = 0.5$. This is because, as φ decreases, the f_t^I threshold increases. A larger threshold leads to more IoT devices being identified as unfair, thus being selected for offloading to the secondary target HAP.

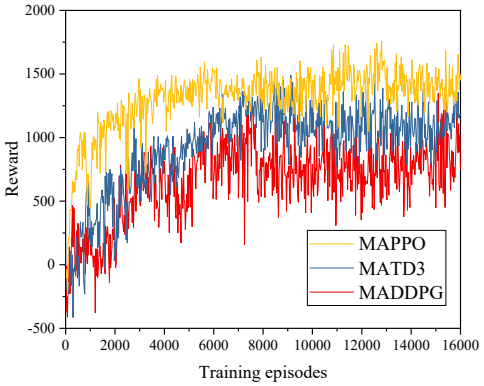


Fig. 3. Cumulative rewards across training episodes

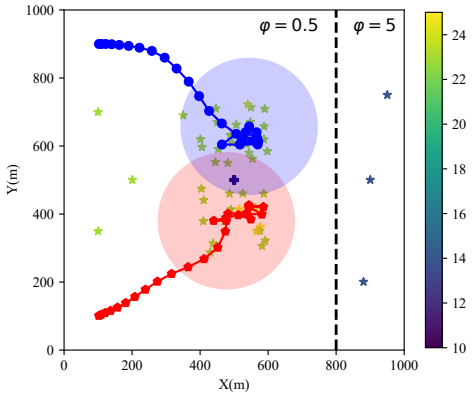


Fig. 4. The heat map of IoT devices and trajectories of UAVs .

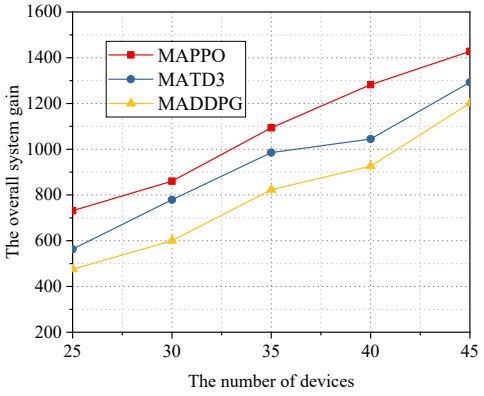


Fig. 5. The Overall gain of the system with different number of IoT devices.

Fig. 5 illustrates the trend of overall system gain over 24 time slots in relation to the number of IoT devices. As the number of IoT devices increases, there is a noticeable linear growth in the overall system gain. Among the strategies considered, the proposed MAPPO-based algorithm performs the best, followed by the MATD3 with the MADDPG showing the poorest performance.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:37:36 UTC from IEEE Xplore. Restrictions apply.

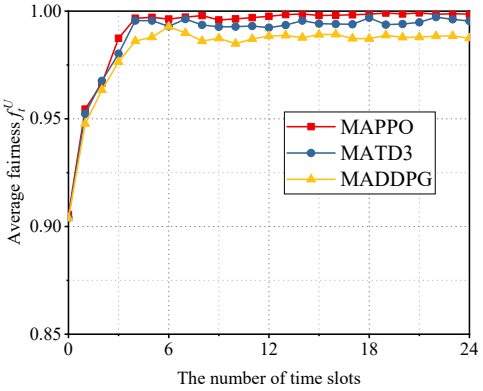


Fig. 6. Average UAV workload fairness f_t^U.

Fig. 6 shows the difference between the three algorithms in terms of the average UAV workload fairness f_t^U. It can be seen that although all three algorithms have fairness metrics close to 1, the MAPPO-based algorithm performs better, which will allow drones to provide services in a more collaborative manner.

V. Conclusion

In this paper, we introduce a hierarchical aerial MEC framework where UAVs and the HAP collaboratively provide computing services for IoT devices. Our optimization objective is to jointly maximize load fairness for each UAV and system gain. To achieve this, we formulate the problem as a MAMDP and propose a MAPPO-based algorithm. Simulation results demonstrate that the proposed approach exhibits significant performance gains compared to benchmark algorithms.

References

[1] D. S. Lakew, A. T. Tran, N. N. Dao, and et al., "Intelligent offloading and resource allocation in heterogeneous aerial access iot networks," *IEEE Internet of Things Journal*, vol. 10, no. 7, pp. 5704–5718, 2022.

[2] H. Kang, X. Chang, J. Mišić, and et al., "Cooperative uav resource allocation and task offloading in hierarchical aerial computing systems: A mappo based approach," *IEEE Internet of Things Journal*, 2023.

[3] M. S. Alam, G. K. Kurt, H. Yanikomeroglu, and et al., "High altitude platform station based super macro base station constellations," *IEEE Communications Magazine*, vol. 59, no. 1, pp. 103–109, 2021.

[4] G. K. Kurt, M. G. Khoshkholgh, S. Alfattani, and et al., "A vision and framework for the high altitude platform station (haps) networks of the future," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 2, pp. 729–779, 2021.

[5] C. Dong, Y. Shen, Y. Qu, and et al., "Uavs as an intelligent service: Boosting edge intelligence for air-ground integrated networks," *IEEE Network*, vol. 35, no. 4, pp. 167–175, 2021.

[6] W. Zhang, L. Li, N. Zhang, and et al., "Air-ground integrated mobile edge networks: A survey," *IEEE Access*, vol. 8, pp. 125 998–126 018, 2020.

[7] X. Dai, Z. Lu, X. Chen, X. Xu, and F. Tang, "Multi-agent rl based jointly trajectory scheduling and resource allocation in noma-assisted uav swarm network," *IEEE Internet of Things Journal*, 2023.

[8] L. Wang, K. Wang, C. Pan, and et al., "Multi-agent deep reinforcement learning-based trajectory planning for multi-uav assisted mobile edge computing," *IEEE Transactions on Cognitive Communications and Networking*, vol. 7, no. 1, pp. 73–84, 2020.

[9] K. Yu, Q. Cui, Z. Zhang, X. Huang, X. Zhang, and X. Tao, "Efficient uav/satellite-assisted iot task offloading: A multi-agent reinforcement learning solution," in *2022 27th Asia Pacific Conference on Communications (APCC)*. IEEE, 2022, pp. 83–88.

[10] L. Wang, K. Wang, C. Pan, and et al., "Deep reinforcement learning based dynamic trajectory control for uav-assisted mobile edge computing," *IEEE Transactions on Mobile Computing*, vol. 21, no. 10, pp. 3536–3550, 2021.

[11] Y. Zhou, C. Pan, P. L. Yeoh, K. Wang, M. Elkashlan, B. Vucetic, and Y. Li, "Secure communications for uav-enabled mobile edge computing systems," *IEEE Transactions on Communications*, vol. 68, no. 1, pp. 376–388, 2019.

[12] Y. Qu, H. Dai, H. Wang, and et al., "Service provisioning for uav-enabled mobile edge computing," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 11, pp. 3287–3305, 2021.

[13] Y. Wang, W. Fang, Y. Ding, and et al., "Computation offloading optimization for uav-assisted mobile edge computing: a deep deterministic policy gradient approach," *Wireless Networks*, vol. 27, no. 4, pp. 2991–3006, 2021.

[14] T. Ren, J. Niu, B. Dai, and et al., "Enabling efficient scheduling in large-scale uav-assisted mobile-edge computing via hierarchical reinforcement learning," *IEEE Internet of Things Journal*, vol. 9, no. 10, pp. 7095–7109, 2021.

[15] S. Zhu, L. Gui, D. Zhao, and et al., "Learning-based computation offloading approaches in uavs-assisted edge computing," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 1, pp. 928–944, 2021.

[16] H. Hao, C. Xu, W. Zhang, S. Yang, and G.-M. Muntean, "Joint task offloading, resource allocation, and trajectory design for multi-uav cooperative edge computing with task priority," *IEEE Transactions on Mobile Computing*, 2024.

[17] W. Zhou, L. Fan, F. Zhou, F. Li, X. Lei, W. Xu, and A. Nallanathan, "Priority-aware resource scheduling for uav-mounted mobile edge computing networks," *IEEE Transactions on Vehicular Technology*, 2023.

[18] A. Mokhtari, M. A. Hossen, P. Jamshidi, and et al., "Felare: Fair scheduling of machine learning tasks on heterogeneous edge systems," in *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)*, 2022, pp. 459–468.

[19] F. Ding, X. Zhang, and L. Xu, "The innovation algorithms for multivariable state-space models," *International Journal of Adaptive Control and Signal Processing*, vol. 33, no. 11, pp. 1601–1618, 2019.

[20] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of ppo in cooperative multi-agent games," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 611–24 624, 2022.

[21] Z. Zong, M. Zheng, Y. Li, and D. Jin, "Mapdp: Cooperative multi-agent reinforcement learning to solve pickup and delivery problems," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 36, no. 9, 2022, pp. 9980–9988.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:37:36 UTC from IEEE Xplore. Restrictions apply.